

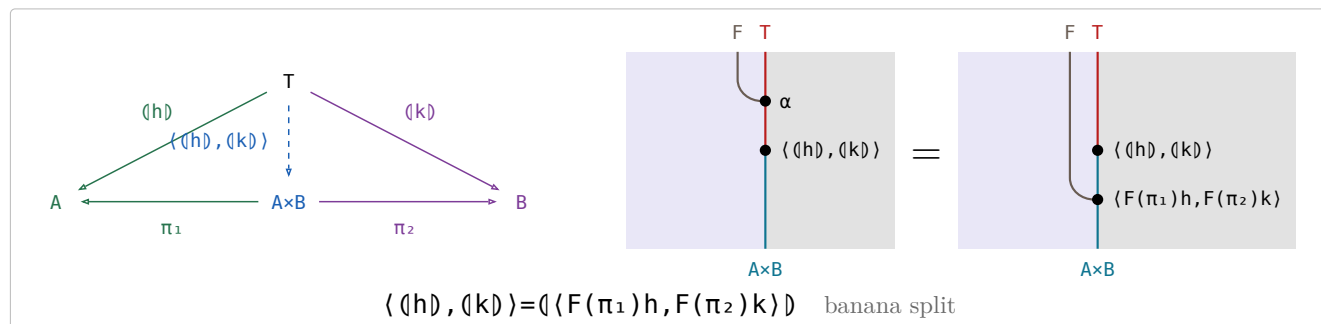
Relation Algebra

§7. Reduce in $\mathcal{S}et$

(7a)

definition	type	note
$\text{listr } A ::= \text{nil} \mid \text{cons}(A, \text{listr } A)$	$\mathcal{A} \rightarrow \mathcal{A}$	The cons-lists over A , the datatype every row below folds.
$\text{sum} = ([\text{zero}, \text{plus}])$	$[A] \rightarrow A$	$\text{plus}(a, b) = a + b$.
$\text{length} = ([\text{zero}, \pi_2 \text{ succ}])$	$[A] \rightarrow \text{Nat}$	π_2 drops the head and keeps the count of the tail, succ adds one for the head.
$\text{average} \triangleq (\text{sum}, \text{length}) \text{div}$	$[\text{Nat}] \rightarrow \text{Nat}$	$\text{div}(m, n) = m/n$, with $\text{div}(0, 0) = 0$ so average is total. Traverses the list twice.
banana-split law $\langle \langle h \rangle, \langle k \rangle \rangle = \langle \langle F(\pi_1)h, F(\pi_2)k \rangle \rangle$	$T \rightarrow A \times B$	Any fork of folds is a single fold, hence one traversal — F the base functor.
what it reduces to $\alpha \langle \langle h \rangle, \langle k \rangle \rangle = F(\langle \langle h \rangle, \langle k \rangle \rangle) \langle F(\pi_1)h, F(\pi_2)k \rangle$	$FT \rightarrow A \times B$	All that $\triangleq$ reduce leaves to check: the fork satisfies the defining equation.
the instance $\langle \text{sum}, \text{length} \rangle = ([\text{zeros}, \text{pluss}])$	$[\text{Nat}] \rightarrow \text{Nat} \times \text{Nat}$	$\text{pluss}(a, (b, n)) = (a + b, n + 1)$, so average runs in one pass.
$\text{preds } n = [n, n-1, \dots, 1]$	$\text{Nat} \rightarrow [\text{Nat}]$	apply the earlier special case to write preds as $\langle k \rangle \pi_1$.

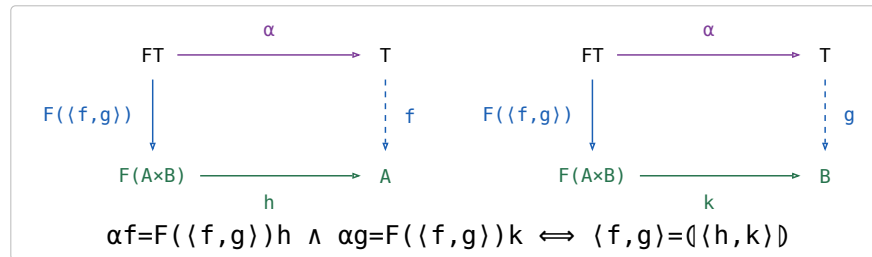
(7b)



§7.1. Fokkinga's mutual recursion theorem

- $\mathbf{F-Alg}(\mathcal{A})$ — the algebras and their homomorphisms — has binary products whenever $\mathcal{A}$ does, and the forgetful $\mathbf{U} : \mathbf{F-Alg}(\mathcal{A}) \rightarrow \mathcal{A}$ creates them: the product of $h : \mathbf{FA} \rightarrow \mathbf{A}$ and $k : \mathbf{FB} \rightarrow \mathbf{B}$ is carried by $\mathbf{A} \times \mathbf{B}$, with structure $\langle \mathbf{F}(\pi_1)h, \mathbf{F}(\pi_2)k \rangle : \mathbf{F}(\mathbf{A} \times \mathbf{B}) \rightarrow \mathbf{A} \times \mathbf{B}$.
- π_1 and π_2 are homomorphisms out of it, and $\langle p, q \rangle : \mathbf{X} \rightarrow \mathbf{A} \times \mathbf{B}$ is a homomorphism **iff** p and q both are — substitute the structure and compare the two forks component by component.
- The banana-split law of (7a) IS that product's universal property read at the initial algebra: $\langle h \rangle$ and $\langle k \rangle$ are the unique homomorphisms to $(\mathbf{A}, h)$ and $(\mathbf{B}, k)$, so their fork is the unique homomorphism into the product, which is $\langle \mathbf{F}(\pi_1)h, \mathbf{F}(\pi_2)k \rangle$.
- Fokkinga's theorem is **strictly more general** and is not that product: in mutual recursion the two arrows leave $\mathbf{F}(\mathbf{A} \times \mathbf{B})$, not $\mathbf{FA}$ and $\mathbf{FB}$, so each sees BOTH components. The product is the case where they factor as $\mathbf{F}(\pi_1)h$ and $\mathbf{F}(\pi_2)k$. What it says: **any algebra on a product carrier is folded by a fork**.

(7.1a)



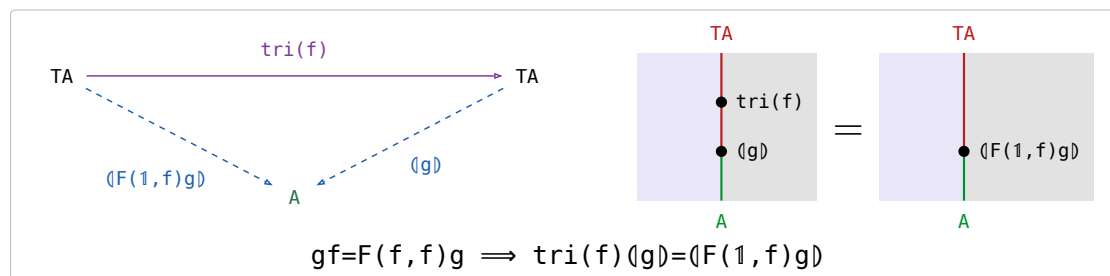
§7.2. Ruby triangles

(7.2a)

stage	definition
informally	$\text{tri}(f)[a_0, a_1, \dots, a_i, \dots, a_n] = [a_0, f(a_1), \dots, f^i(a_i), \dots, f^n(a_n)]$
for cons-lists	$\text{tri}(f) = \langle [\text{nil}, (1 \times \text{listr}(f)) \text{ cons}] \rangle$
with the base functor named	$\mathbf{F}(\mathbf{A}, \mathbf{B}) = 1 + \mathbf{A} \times \mathbf{B}$, $\alpha = [\text{nil}, \text{cons}]$, so $\text{tri}(f) = \langle \mathbf{F}(1, \text{listr}(f)) \alpha \rangle$
abstractly	$\mathbf{F}$ a bifunctor with initial type $(\alpha, \mathbf{T})$: $\text{tri}(f) = \langle \mathbf{F}(1, \mathbf{T}(f)) \alpha \rangle$

For the definition to make sense $f : \mathbf{A} \rightarrow \mathbf{A}$ is required, and then $\text{tri}(f) : \mathbf{TA} \rightarrow \mathbf{TA}$.

(7.2b)



Horner's rule is Horner's rule, generalised from cons-lists to any initial type. Fusion reduces it to $\mathbf{F}(1, \mathbf{T}(f)) \alpha (g) = \mathbf{F}(1, (g)) \mathbf{F}(1, f) g$.

§7.3. Depth of a tree

(7.3a)

the datatype	$\text{tree } A ::= \text{tip}(A) \mid \text{bin}(\text{tree } A, \text{tree } A)$, base functor $F(A, B) = A + B \times B$, initial type $([\text{tip}, \text{bin}], \text{tree})$
the fold	$\langle [g, h] \rangle$ is the unique f with $f(\text{tip}(a)) = g(a)$ and $f(\text{bin}(x, y)) = h(f(x), f(y))$
$\text{tree}(f)$	$\text{tree}(f) = \langle F(f, 1) \mid [\text{tip}, \text{bin}] \rangle$; pointwise $\text{tree}(f)(\text{tip}(a)) = \text{tip}(f(a))$ and $\text{tree}(f)(\text{bin}(x, y)) = \text{bin}(\text{tree}(f)(x), \text{tree}(f)(y))$
max	$\text{max} = \langle [1, \text{bmax}] \rangle$, where $\text{bmax}(a, b)$ is the larger of a and b
depths	$\text{depths} = \text{tree}(\text{zero}) \text{ tri}(\text{succ})$ — replaces every tip by its depth in the tree, zero the constant function returning 0 and succ the successor
depth	$\text{depth} = \text{depths } \text{max}$